

The Rails Way とは？

-原則とその向こうへ-

作成者: 石田 隼人

最終更新日: 2025年11月14日

英語版は [こちら](#)

自己紹介

- 各種アカウント
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- 職業: ソフトウェアエンジニア
- 趣味
 - 言語学
 - カラオケ
 - 音楽鑑賞
 - 映画鑑賞
 - 猫飼育



免許 / 資格

- 英語
 - TOEIC® Listening & Reading 915点: 2019年12月 取得
- エンジニアリング
 - 情報セキュリティマネジメント: 2017年11月 取得
 - 応用情報技術者: 2017年06月 取得
 - 基本情報技術者: 2016年11月 取得
 - IT パスポート: 2016年04月 取得
- その他
 - 珠算2級: 2002年06月 取得
 - 暗算3級: 2001年02月 取得

スキルスタック

- 自然言語
 - 日本語: 母語
 - 英語: ビジネスレベル
- 開発
 - **バックエンド: Ruby(Ruby on Rails), Python**
 - フロントエンド: TypeScript + React.js, TypeScript + Vue.js
 - **データベース: MySQL, PostgreSQL, MongoDB**
 - **アーキテクチャー: モノリス, モジュールモノリス**
 - ホスティング: AWS ESK
 - バージョン管理: Git, GitHub
 - **CI/CD: GitHub Actions, ArgoCD**
 - 監視: Datadog, Sentry

職歴

期間	業種	職種	業務内容
2021年08月- Present	EdTech SaaS	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • CI/CD 運用・保守 • 週次リリースマネージャー • トラブル対応 • ドキュメンテーション • 技術負債解消 • 技術ブログ記事執筆 • LT 登壇
2020年02月- 2020年12月	チャットボット系 SaaS	バックエンド エンジニア	• バックエンド開発 • ドキュメンテーション • 技術負債解消 • 代替 API 検証

職歴

期間	業種	職種	業務内容
2018年06月 - 2020年01月	受託開発	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • トラブル対応 • QA • 技術ブログ記事執筆
2016年04月 - 2018年01月	SES	システムエンジニア	• 全社アカウント管理者 • Windows Server 運用・保守 • セキュリティ管理者助手 • 翻訳・通訳

国際交流活動

- 大学時代
 - 英語学ゼミ(マスメディア英語)
 - 国際交流サークル兼部(2年次)
 - [内閣府主催国際交流プログラム](#)(2013年 - 2016年)
 - 日本語学の授業の S 単位取得(最終年次)
- 海外生活
 - オーストラリアでのワーキングホリデー(2014年04月 - 2015年03月)
 - 2ヶ月間のシドニーの語学学校通学
 - 6ヶ月間の [Hamilton Island Resort](#) での就労
 - 1ヶ月間の NSW 州の [St Ives High School](#) での日本語教師アシスタントボランティア活動
- その他活動
 - 英語での日々の日記(2014年04月 - 現在)
 - [Sunrise Toastmasters Club](#) 参加(2017年02月 - 2018年03月)
 - [Vital Japan](#) 参加(2018年01月 - 2019年07月、2022年10月 - 2023年02月)
 - 英語自己学習
 - Ruby 関連カンファレンス参加

Agenda

1. 何故興味を持ったか？
2. "The Rails Way" の哲学 -MVC vs. 多層構造型-
3. 主要な責務 -MVC vs. 多層構造型-
4. 拡張型 The Rails Way の主要構成要素(多層構造型)
5. 長所・短所
6. 漸進的な多層構造の取入れ戦略
7. アンケート回答
8. まとめ
9. 参考文献

1. 何故興味を持ったか？

The logo for RAILS, featuring a stylized white train icon on the left and the word "RAILS" in a bold, white, sans-serif font on the right, all set against a red circular background.

1. 何故興味を持ったか？

Rails アプリケーションのカオスなエコシステム整備のつらみを経験

- 入れ子になったドメイン名前空間: システム種別とアカウント種別
 - ※ [次スライド](#) にて図解
- ロジックや実装場所の一貫性のなさ
- 冗長な多層構造
 - → *CQRS, Forms, Validators, Decorators, Serializers* など
- バックエンド・フロントエンド間の適切に管理されていないルーティング
- モジュールモノリスではない「相乗りモノリス」

→ 「The Rails Way とは何か」 が全く分かん!!

1. 何故興味を持ったか？

モジュールモノリス vs. 相乗りモノリス

```
monorepo/
├─ backend-app-1/
│  └─ app/
│     ├── assets/
│     ├── channels/
│     ├── controllers/
│     ├── helpers/
│     ├── jobs/
│     ├── mailers/
│     ├── models/
│     └─ views/
│  └─ bin/
│     ├── rails
│     ├── rake
│     └─ setup
│  └─ config/
│     ├── environments/
│     ├── initializers/
│     ├── locales/
│     ├── application.rb
│     ├── boot.rb
│     ├── database.yml
│     ├── routes.rb
│     └─ storage.yml
├─ frontend-app-1/
│  ├── public/
│  └─ src/
│     ├── assets/
│     ├── components/
│     ├── pages/
│     ├── hooks/
│     ├── context/
│     ├── services/
│     ├── utils/
│     ├── store/
│     ├── routes/
│     ├── layouts/
│     ├── types/
│     ├── __tests__/
│     ├── App.jsx
│     ├── App.css
│     ├── index.jsx
│     └─ index.css
│  ├── .env
│  ├── .gitignore
│  ├── package.json
│  ├── package-lock.json
│  ├── README.md
│  └─ vite.config.js
├─ backend-app-2/
├─ frontend-app-2/
└─ k8s-dev/
```

VS.

```
omnirepo/
├─ app/
│  ├── javascript/
│  │  └─ teacher_app/
│  ├── commands/
│  ├── controllers/
│  │  ├── app1/
│  │  ├── app2/
│  │  ├── account_type_1/
│  │  ├── account_type_2/
│  │  └─ account_type_3/
│  ├── app3/
│  ├── decorators/
│  ├── errors/
│  ├── factories/
│  ├── forms/
│  ├── helpers/
│  ├── mailers/
│  ├── models/
│  └─ workers/
```


1. 何故興味を持ったか？

とある Rubyist の方が [Kaigi on Rails 2025](#) にて投稿した内容、それが...

"rails way とは何なのか?"



1. 何故興味を持ったか？



1. 何故興味を持ったか？

同意する理由

- 公式な "The Rails Way" の定義がない
 - → 十人十色の解釈
- **MVC** エコシステム・**CoC** 哲学のいずれか、もしくは両方を指している？
 - → 曖昧模糊で話が噛み合わない
- 「おまかせ」とも違う？
 - → 作者の DHH すら "The Rails Way" という表現を使っていない

→ もはや "The Rails Way" という合言葉では Rails アプリケーションのアーキテクチャについて合意を取れない

1. 何故興味を持ったか？

✗ Now is not the time to cry

✓ **Now's the time to find out why "The Rails Way"**

Live Forever - Oasis

2. "The Rails Way" の哲学 -MVC vs. 多層構造型-



2. "The Rails Way" の哲学 -MVC vs. 多層構造型-

従来の MVC アーキテクチャー

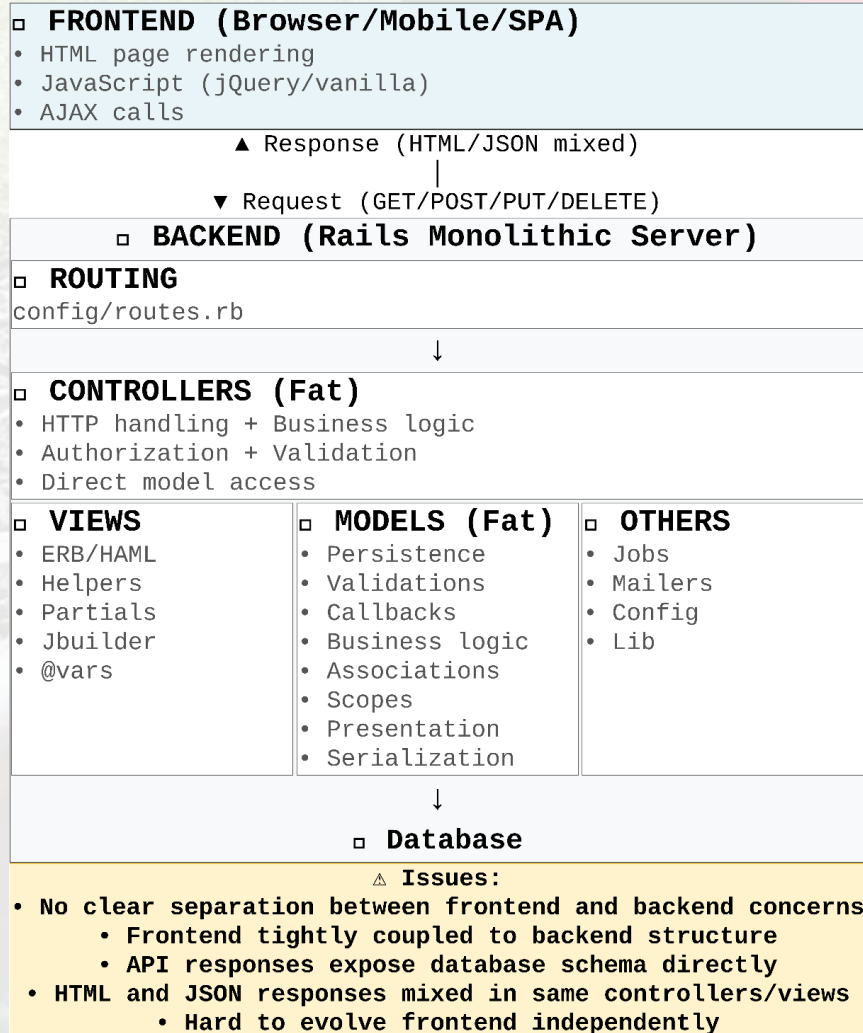
- 設定より規約
 - 機能を出来る限り推論出来る仕組み
 - オブジェクト名やプロジェクトの構造に基づく
 - セットアップよりも製品開発に集中出来るように
 - 数(単数形/複数形)や格変化(e.g. I-my-me)などの文法上の屈折
 - e.g. **User** モデルはデータベースの **users** テーブルに相当
- 3層構造: MVC
 - **Models** (ActiveRecord)
 - **Views** (ERB HTML テンプレート)
 - **Controllers** (データフロー制御)

2. "The Rails Way" の哲学 -MVC vs. 多層構造型-

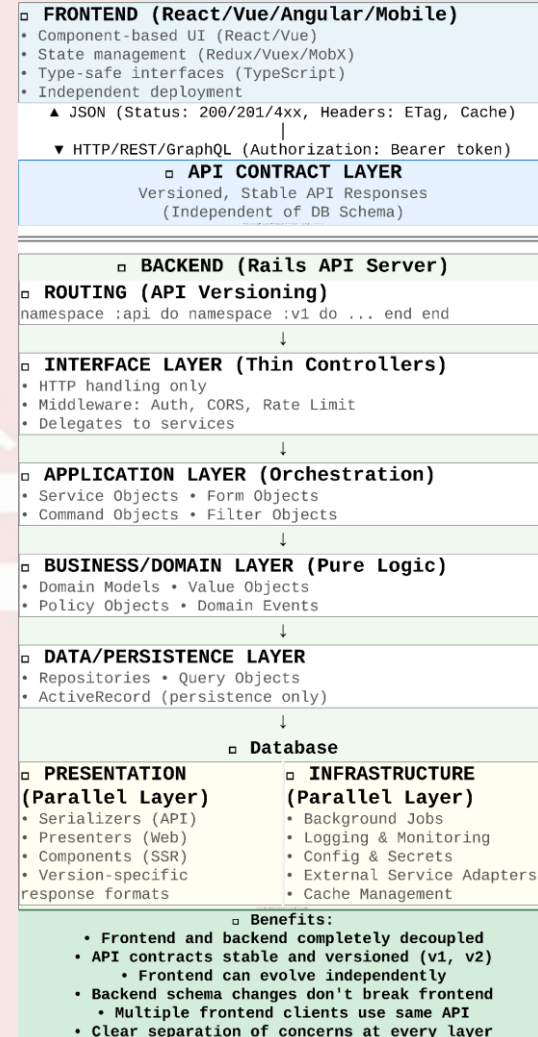
拡張/多層構造型 アーキテクチャー

- 構造的な抽象化: 役割の分離
 - ロジックのカプセルか
 - 詳細な実装よりも外側の振舞いに着目
 - → オブジェクト指向
 - 1対1 の単一かつ明確な責務の割当て
 - 薄いコントローラーとモデル
- 5つ (かそれ以上の) 役割に特化した層:
 - プレゼンテーション層
 - ビジネス/ドメイン層
 - アプリケーション/サービス層
 - データ/永続層
 - インフラ

2. "The Rails Way" の哲学 -MVC vs. 多層構造型-



VS.



3. 主要な責務 -MVC vs. 多層構造型-



3. 主要な責務 -MVC vs. 多層構造型-

責務	MVC	多層構造型
プレゼンテーション	• <code>app/controllers/*_controller.rb</code>: インスタンス変数 • <code>app/helper/*_helper.rb</code>: カスタムヘルパー • ActiveModel::Serializers::JSON#as_json	• <code>app/serializers/*_serializer.rb</code> • <code>app/view/components/</code> • <code>app/view/presenters/</code>
フォーム操作	• <code>app/controllers/*_controller.rb</code>: Strong parameters • <code>app/model/*.rb</code>: バリデーション	<code>app/forms/*_form.rb</code> (コンテキスト準拠)
ビジネスロジック	Scattered Everywhere • <code>app/controllers/*_controller.rb</code>(concerns を含む) • <code>app/model/*.rb</code>(concerns を含む) • <code>app/views/*.html.erb</code>(⚠ アンチパターン)他	• <code>app/model/*.rb</code> • <code>app/services/*_service.rb</code>
認証	• <code>app/controllers/*_controller.rb</code>: Callbacks • <code>app/model/*.rb</code>: Attributes	<code>app/policies/*_policy.rb</code> (single source of truth)
データ接続	Executed Everywhere • <code>app/controllers/*_controller.rb</code>(concerns を含む) • <code>app/model/*.rb</code>(concerns を含む) • <code>app/views/*.html.erb</code>(⚠ アンチパターン)他	• <code>app/commands/*_command.rb</code> • <code>app/repositories/*_repository.rb</code> • <code>app/queries/*_query.rb</code>

4. 拡張型 The Rails Way の主要構成要素(多層構造型)



4. 拡張型 The Rails Way の主要構成要素(多層構造型)

4-1. プレゼンテーション 層

- [View Components](#): 明確なインターフェース
- [Presenters](#): コンテキストを意識したフォーマット実行
- [Serializers](#): API 通信、バージョニング



RAILS

4. 拡張型 The Rails Way の主要構成要素(多層構造型)

4-2. アプリケーション/サービス層

- [Service Objects](#): 単一のビジネス関連の操作
- [Form Objects](#): 複数モデルの操作、コンテキストのバリデーション
- [Filter Objects](#): ユーザー入力



4. 拡張型 The Rails Way の主要構成要素(多層構造型)

4-3. ビジネス/ドメイン層

- [Domain Models](#): 多様な振舞い、永続処理の実体の消失
- [Policy Objects](#): 認証規則
- [Value Objects](#): 変更不可なドメインの概念

4. 拡張型 **The Rails Way** の主要構成要素(多層構造型)

4-4. データ/永続層

- [Repositories](#): データ接続
- [Query Objects](#): 再利用可能な複雑なクエリ
- [ActiveRecord](#): 永続処理のみの実装



RAILS

5. 長所・短所

The logo features a stylized white train icon on the left, composed of a semi-circle and a vertical line, followed by the word "RAILS" in a bold, white, sans-serif font. The entire logo is centered within a large, semi-transparent red circle that is itself centered on a grey rectangular background. This background is superimposed over a faded image of a railway track receding into the distance.

5. 長所・短所

従来の MVC アーキテクチャー

- 👍 高速な開発: 組み込みコマンドやジェネレーターの利便性
- 👍 設定より規約: 余計な設定ファイル要らずの高い生産性への注力
- 👍 Gem の豊かなエコシステム: [中核業務](#) に最大限注力
- 👍 豊富な資源: サポート熱心な大きなコミュニティからの助力、ドキュメント、チュートリアルやその他情報源へのアクセスの良さ
- 👍 組み込みのセキュリティ関連機能: XSS, CSRF, and SQL injection のようなウェブの脆弱性対応
- 👍 最善策の提供: コードの保守性と信頼性の担保
- 👍 プログラマの喜び: 「好きこそものの上手なれ」

5. 長所・短所

従来の MVC アーキテクチャー

- 🖐️ **初学者にとっての険しい学習曲線:** 「魔法」のメカニズムを知る必要がある
- 🖐️ **大規模開発における柔軟性:** 拡張型 "The Rails Way" の最適解を見つけるのが難しい
- 🖐️ **大規模開発におけるパフォーマンス:** 優れた並列処理やレスポンスの遅延の少なさでは最適な選択ではない
- 🖐️ **デプロイ作業の複雑さ:** アセットプリコンパイルやデータベースマイグレーション、非同期ジョブなど DevOps の知見を必要とするコンポーネントの管理
- 🖐️ **実行中の処理の実態:** 難解なメカニズムによるデバッグやカスタマイズ体験の悪化

5. 長所・短所

拡張/多層構造型 アーキテクチャー

- 👍 **保守性**: 分かりやすい分割単位、コードの特定のし易さ
- 👍 **テスト容易性**: 分離された層、フレームワークの疎結合
- 👍 **拡張性**: 線形成長、独立層
- 👍 **柔軟性**: 実装の入替え、コンテキストに即した振舞い
- 👍 **明晰さ**: コードがビジネスロジックの仕様書
- 🤔 **初手の複雑さ**: 数多のファイル、前倒しの設計、険しい学習曲線
- 🤔 **遅い立上り**: scaffold のサポートなし、単純機能のオーバーヘッド
- 🤔 **チームの共通認識**: 全員の例外なき設計パターンへの合意が必要
- 🤔 **デバッグ**: スタックとレースの肥大化、経路の複雑化
- 🤔 **移行コスト**: 既存アプリの継続的なリファクタリング労力

6. 漸進的な多層構造の取入れ戦略

The logo features the word "RAILS" in a bold, white, sans-serif font. To the left of the text is a stylized white graphic consisting of a semi-circle and a dashed line, resembling a train track or a signal. The entire logo is centered within a large, semi-transparent red circle that is itself centered on the page. The background of the slide is a blurred photograph of a railway track with gravel and green vegetation.

6. 漸進的な多層構造の取入れ戦略

✓ 適切なケース - ROI(投資対効果)が十分

- 実用最小限の製品開発(MVP)の段階を超えた場合
- 3人以上の開発者がいるチームが複数ある場合
- ビジネス/ドメインが複雑な場合
- 長期保守が必要な場合
- 複数の API クライアントを持つサービスの場合
- パフォーマンスのボトルネック MVC アーキテクチャにある場合

6. 漸進的な多層構造の取入れ戦略

✖ 不適切なケース - ROI(投資対効果)が乏しい

- 単純な CRUD アクションだけで完結する場合
- プロトタイプ/実用最小限の製品開発(MVP) の段階である場合
- 1-2人で構成されたチームしかない場合
- 短期のプロジェクトの場合

RAILS

6. 漸進的な多層構造の取入れ戦略

最善策

1. **MVC より始めよ:** 新規プロジェクトは従来のアーキテクチャーで！
2. **ペインの閾値に刮目せよ:**
 - 200行以上の複数モデル
 - 複雑なワークフローを処理する複数のコントローラー
 - 遅くて保守が利かないテスト(必要な挙動の検証が抜け漏れてしまう場合も含む)
3. **漸進的に抽出せよ:** 必要な分だけ特定領域に絞ってリファクタリングを！
4. **規約に準拠せよ:** Rails のイディオムを適切に管理(例. ActiveRecord)
5. **美味しいところ取りせよ:** シンプルな箇所は従来の MVC、複雑な箇所は多層構造の上手な適用を！

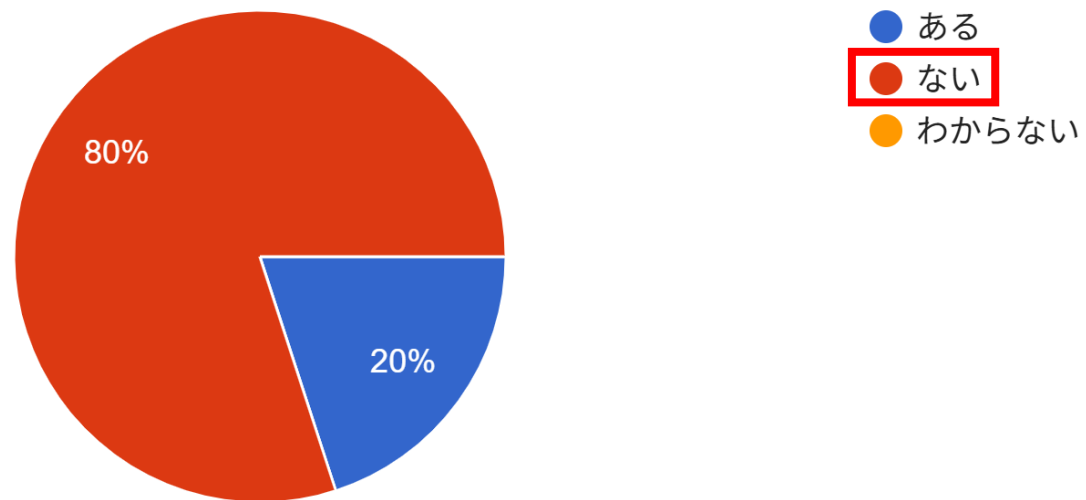
7. アンケート回答



7. アンケート回答

"The Rails Way" を合言葉にアーキテクチャーの合意形成に失敗した経験はありますか？

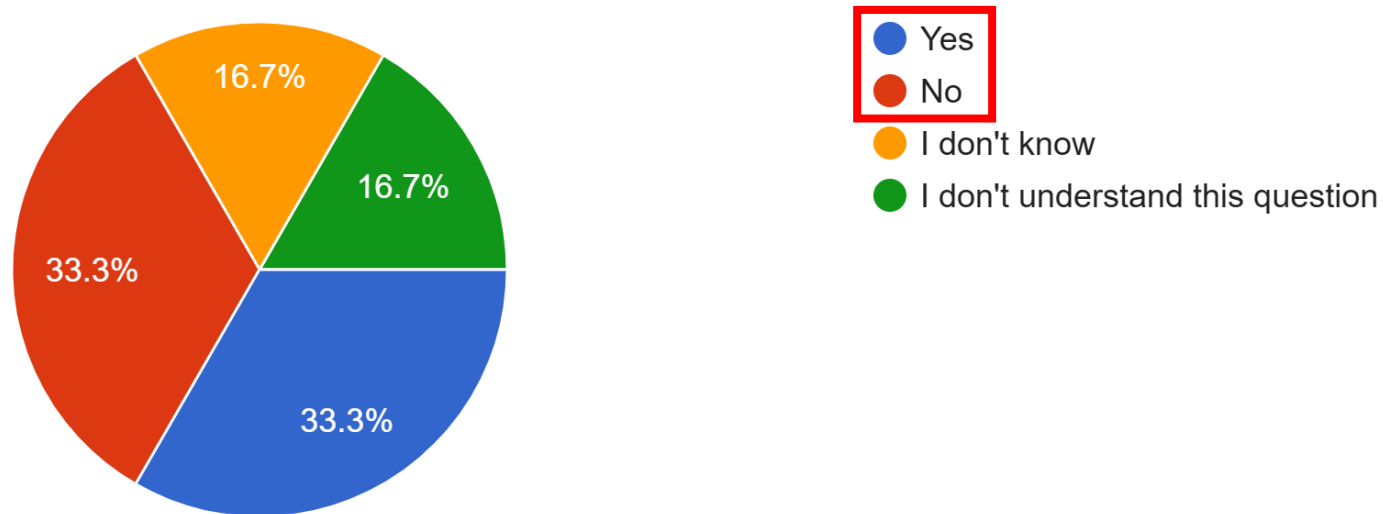
5 responses



7. アンケート回答

Have you ever failed to arrive at an agreement with your Rails app architecture with the keyword "The Rails Way"?

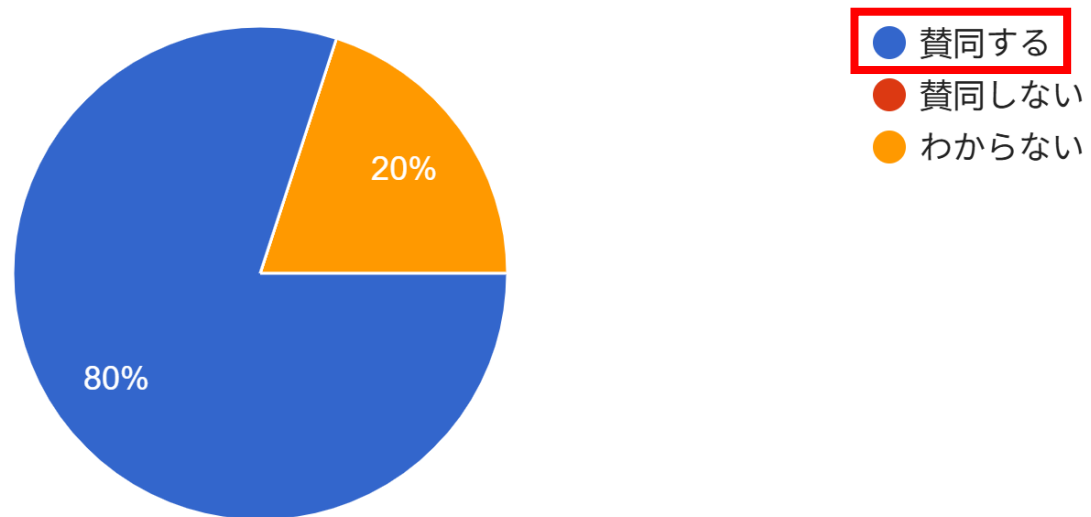
6 responses



7. アンケート回答

Scaffold のエコシステムが「狭義の The Rails Way」という言説がありますが、賛同しますか？ ※
最低限の MVC と CoC

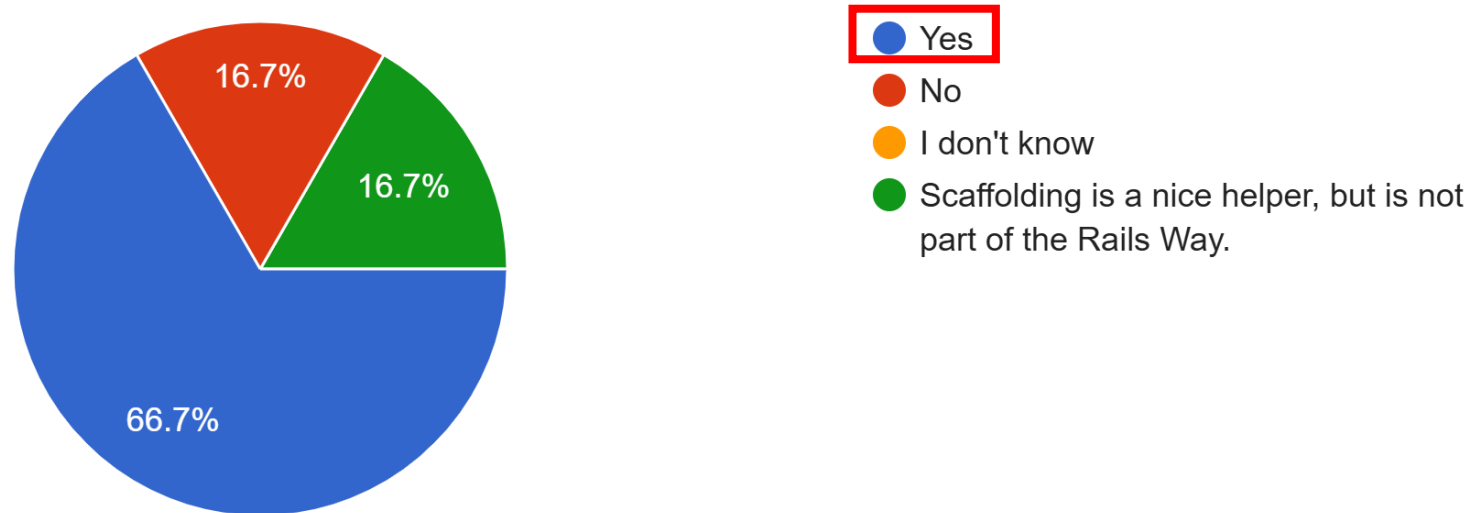
5 responses



7. アンケート回答

Here is the concept that the ecosystem generated by `bin/rails g scaffold ...` is treated as "The Rails Way in a narrow sense". Do you agree with it? * e.g.) MVC, CoC, Responsibility segregation

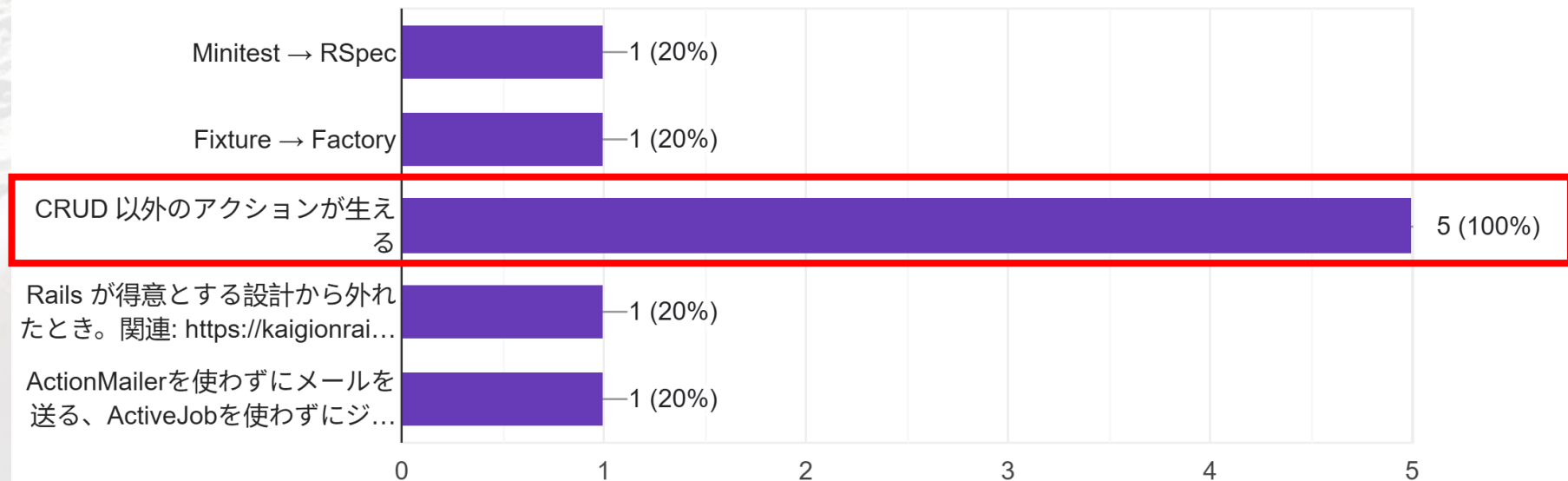
6 responses



7. アンケート回答

「狭義の The Rails Way」の観点で、以下のエコシステムの中で置き換えられたら「それはもはや The Rails Way ではない」と考えるものを全て選んで下さい。

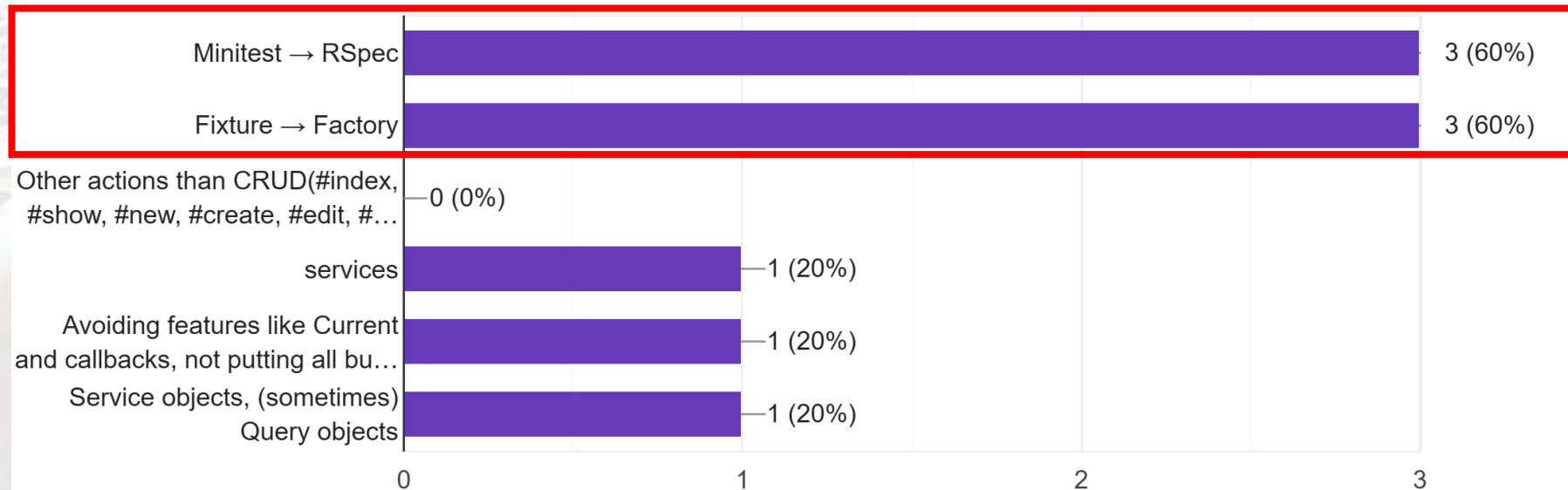
5 responses



7. アンケート回答

Among the following components, what gets your Rails app out of "The Rails Way in a narrow sense"?

5 responses

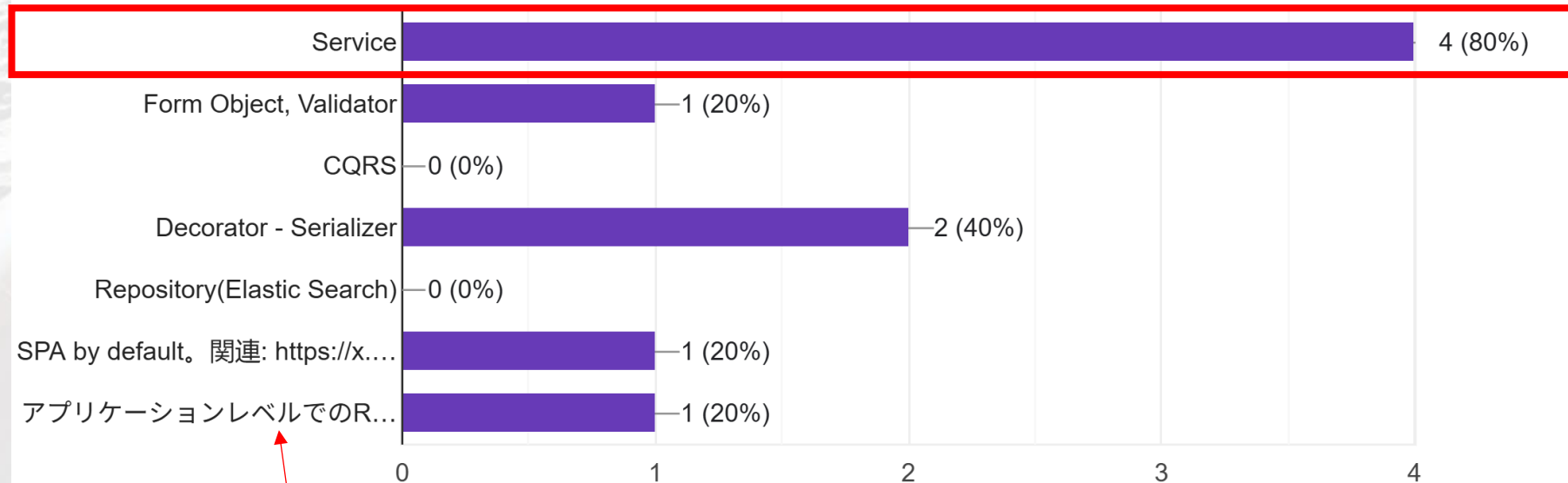


7. アンケート回答

「広義の The Rails

Way」において、あなたが最も苦しめられたエコシステムを全て選んで下さい。

5 responses

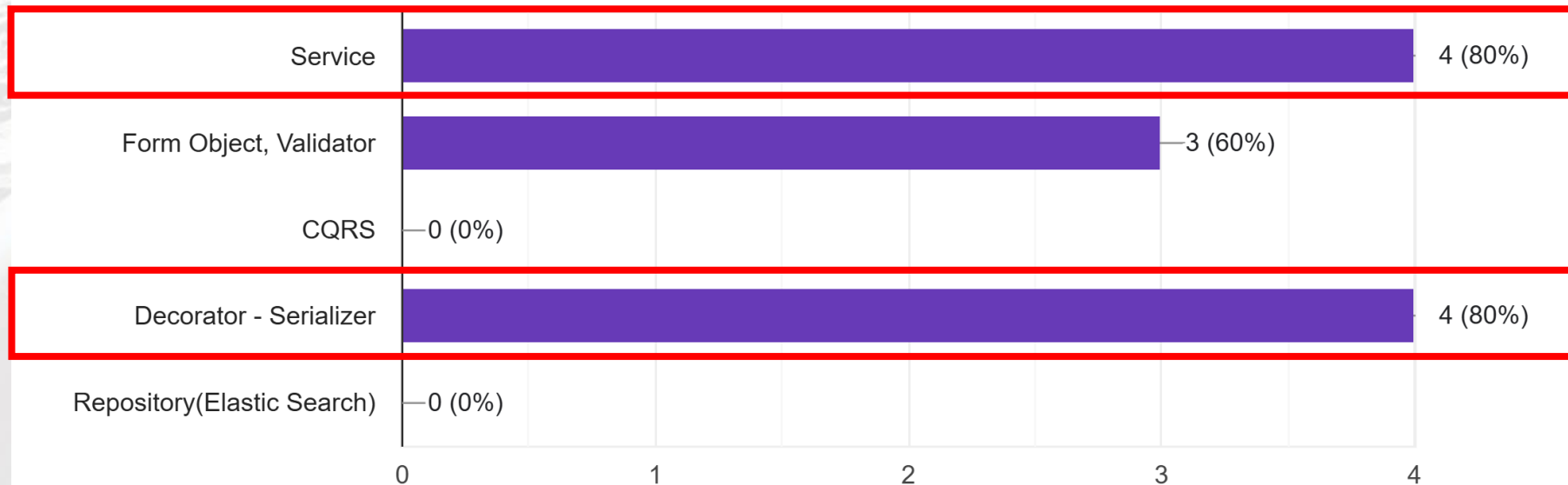


アプリケーションレベルでの Rails っぽい CoC の独自実装の提供 (method_missing の多用を含む)

7. アンケート回答

What have you got in trouble with among the following ecosystem of "The Rails Way in a broader sense"?

5 responses



7. アンケート回答

Q. あなたが思う "The Rails Way" の条件を自由に記述して下さい - 海外出身 Rubyists の回答

- 「サービス」の名の下何でも詰め込むんだり [dry-rb](#) を多用するのは悪手で、ActiveRecord とビジネスロジックは密結合し得ることを受け入れる
- Rails が持つ最適解に従いコミュニティの好みに合わせること。個人的には The Rails Way から派生したくなる場面はあるが成長過程にあるチームではそれを避けて最適解に従う
- Rails はレゴのようなもので、持っているブロックで自分なりの宇宙船を作れば良い
- 抽象化ゆえに Rails に関する意思決定は最適でない場合がある

7. アンケート回答

Q. あなたが思う "The Rails Way" の条件を自由に記述して下さい - 日本人 Rubyists の回答(一部抜粋・加工)

- イレギュラーなものはイレギュラーであると皆が自覚できていればそれはまだ The Rails Way を保持できているとしてもよいのかなとも思う
- Rails new と scaffold で作られるファイルとディレクトリ構造以外は全て "The Rails Way" ではないとすると、個人的には考えやすい
- Rails の用意した CoC に従い RESTful なリソース設計を原則としてビジネスロジックを Model 層に集約すること
- スタートアップの最初期は価値を届けられないと終わる => 製品の価値提供の最大化に実用的なエコシステムが正義

8. まとめ

The logo for RAILS, featuring a stylized white train track curving upwards and to the right, with the word "RAILS" in bold, white, sans-serif capital letters.

8. まとめ

- **従来の MVC:** アイデアから実装までのスピードが MVP に最適
 - → 狭義の "The Rails Way"
- **多層構造化:** 規模拡大時の境界の明確化と保守・テスト容易性の担保
 - → 広義の "The Rails Way"
- **漸進的な移行:** シンプルに始め複雑化した時に初めて多層化を導入
- **長期保守:** 現実を感じるペインを手掛かりにリファクタリング
- **狭義の "The Rails Way":** 大半が **CoC + MVC** の図式に合意
- **広義の "The Rails Way":** 大半が **Service Objects** にペインを感じていた
- **"The Rails Way" から外れる時:** 海外出身 Rubyists にとっては**デフォルトのエコシステムを入れ替えた**時で、日本人 Rubyists にとってのそれが **CRUD 以外のアクションが生えた**時であることと好対照

8. まとめ

長い話をまとめると...

"Think Less, Create More!"こそ The Rails Way!

9. 参考文献

The logo features a stylized white train engine icon on the left, composed of a semi-circle and a dashed line. To its right, the word "RAILS" is written in a bold, white, sans-serif font. The entire logo is centered within a large, semi-transparent red circle that is itself centered on the slide.

9. 参考文献

- Vladimir Dementyev、[Layered Design for Ruby on Rails Application](#)、バーミンガム、2023年
- 石田隼人、[Layered Design for Ruby on Rails Application - Full Summary](#)、東京、2025年
- 石田隼人、[Layered Design for Ruby on Rails Application - Concise Summary](#)、東京、2025年
- ["The Rails Way" Survey - EN](#)
- ["The Rails Way" Survey - JP](#)
- [Statistics of "The Rails Way" Survey](#)
- TechRacho、[「Ruby: Dry-rb gemシリーズのラインナップと概要」](#)、東京、2017年

